

Shopify-Shop erstellen

Der vollständige Prozess von der Grundeinrichtung bis zur Übergabe an den Kunden — sieben Skills, 31 Phasen, zwölf Freigabepunkte.

1 · Grundeinrichtung

2 · Aufbau

3 · Produkte

4 · Abnahme & Übergabe

Inhalt

1	Das Prinzip	Warum der Ablauf fest ist
2	Die Kette im Überblick	Vier Stufen, sieben Skills
3	Vorbereitung	Installation, Custom App, Token
4	Grundeinrichtung	shopify-settings · 7 Phasen
5	Aufbau im Theme	shopify-design · 11 Phasen
6	Bilder	Hero, Kategoriebanner, Grenzen
7	Produktmigration	shopify-migration · 7 Phasen
8	Abnahme	shopify-abnahme · UX, CRO, Recht
9	Die Übergabe	Uebergabe.md und der Livegang
10	Anhang	Alte Checkliste, Fallen, Befund-Format

Das Prinzip

Der Ablauf ist streng vorgegeben und immer gleich. Den größten Teil macht die KI, ein Mensch kontrolliert an festen Punkten und gibt frei.

Drei Eigenschaften, die den Prozess tragen

Fester Ablauf

Jeder Skill hat nummerierte Phasen in fester Reihenfolge.

Zwei Mitarbeiter, die denselben Shop bauen, kommen an denselben Punkten vorbei. Was in Phase 3 gehört, passiert nicht in Phase 6 — sonst muss man es später bei hunderten Produkten nachziehen.

Freigabe vor jedem Eingriff

Zwölf Freigabepunkte über den ganzen Prozess.

Jeder schreibende Schritt verlangt ein getipptes JA. Lesende Prüfungen laufen jederzeit. So kann niemand versehentlich hundert Produkte überschreiben.

Nichts erfinden

Was niemand wissen kann, wird zur Rückfrage.

Lieferzeiten, Preise, Maße, Herstelleradressen, Rechtstexte. Die Skripte setzen [RÜCKFRAGE ...] ein, spätere Phasen finden sie wieder und blockieren die Veröffentlichung.

Ungeprüft ist nicht in Ordnung

Fehlende Berechtigung heißt unbekannt, nicht erledigt.

Konnte ein Punkt nicht geprüft werden, steht er als eigener Abschnitt in der Übergabe. Ein stillschweigend abgehakter Punkt ist die gefährlichste Zeile im ganzen Dokument.

Was Claude in diesem Prozess nicht tut

Diese Grenzen sind in den Skills festgeschrieben und keine Auslegungssache.

Grenze	Warum
Keine Rechtstexte	AGB, Widerruf, Datenschutz, Impressum kommen vom Kunden, seinem Anwalt oder einem Dienst wie der IT-Recht Kanzlei. Auch kein Entwurf — ein Platzhaltertext, der live geht, ist genau der Fall, den die Abmahnung trifft.
Keine erfundenen Fakten	Maße, Gewichte, Preise, Lieferzeiten, Herstelleranschriften, Zielgruppenzuordnungen. Recherchiertes wird mit Quelle vorgelegt und bestätigt.
Keine Zahlungsanbieter	Dort werden Bankdaten und Ausweisdokumente verlangt. Ebenso keine Domainkäufe und keine Tarifwahl.
Kein Token im Chat	Zugangsdaten gehören nicht in den Chat, in E-Mails, in Tickets oder auf Screenshots. Claude liest die Datei selbst.

Die App erstellt der Mensch

Claude erzeugt keine Zugangsdaten. Der Weg dorthin steht in der Anleitung, gegangen wird er im Shop-Admin.

Nie auf dem aktiven Theme

Alle Theme-Skripte verweigern das MAIN-Theme und verlangen ein Duplikat.

Der häufigste Fehler beim Einstieg

Die Reihenfolge überspringen. Wer die Produkte vor dem Aufbau importiert, merkt erst bei dreihundert Datensätzen, dass ein Metafeld im Theme gar nicht ankommt. Genau dafür steht das Demo-Produkt in Stufe 2:

ein

Produkt, an dem die ganze Kette geprüft wird, bevor die echten kommen. Und wer die Abnahme an einem halbfertigen Shop laufen lässt, erzeugt Befunde, die sich beim Weiterbauen von selbst erledigen.

Die Kette im Überblick

Vier Stufen. Jede setzt voraus, dass die vorige abgenommen ist.

STUFE 1	STUFE 2	STUFE 3	STUFE 4
shopify-settings Rahmen: Stammdaten, Richtlinien, Versand, Steuern, Sprachen, Metafeld-Definitionen	shopify-design Aufbau: Copy, Website-Konzept, Gestaltung, Bilder, Sections, Demo-Produkt	shopify-migration Produkte: Steckbrief, Importvorlage, Prüfung, Import, Abnahme	shopify-abnahme Prüfung: UX, CRO, Recht — dann die Übergabe-Checkliste

Sieben Skills in vier Plugins

Skill	Phasen	Freigaben	Ergebnis
shopify-settings	7	2	Einrichtungsstand.md — was steht, was fehlt, was unklar blieb
shopify-design	11	3	Shop-Copy, Website-Konzept, Theme-Duplikat, geprüfetes Demo-Produkt
shopify-migration	7	5	Produkte im Shop, Soll-Ist-Vergleich nach dem Import
shopify-abnahme	4	2	Uebergabe.md — die Livegang-Liste
shopify-ux	4	1	befunde/ux.json
shopify-cro	4	1	befunde/cro.json
shopify-recht	4	1	befunde/recht.json

Einzelnen oder gesamt

Die drei Prüf-Skills laufen auch für sich, wenn nur ein Blickwinkel gefragt ist. Sie schreiben in dieselben Befund-Dateien, sodass eine spätere Gesamt-Abnahme darauf aufsetzt statt neu anzufangen.

So wird ein Skill aufgerufen

Terminal im Kundenordner öffnen, `claude` starten, in normaler Sprache sagen, was ansteht. Claude wählt den passenden Skill selbst.

```
> Ich möchte einen Shopify-Shop einrichten.
> Ich möchte Produkte in einen Shopify-Shop migrieren.
> Setz mir das Konzept im Theme um.
> Mach eine Abnahme — kann der Shop live gehen?
```

Es muss keine Datei geöffnet und kein Befehl getippt werden. Die Skript-Aufrufe in dieser Dokumentation stehen zur Nachvollziehbarkeit hier — im Alltag ruft Claude sie selbst auf und erklärt die Ergebnisse im

Chat.

Vorbereitung

Einmal je Mitarbeiter: Plugins installieren. Einmal je Kunde: Custom App anlegen und den Token ablegen.

Plugins installieren — einmal je Mitarbeiter

Im Terminal `claude` starten und nacheinander eingeben:

```
/plugin marketplace add mahun97/hdc-shopify-migration

/plugin install shopify-settings@hdc-digital
/plugin install shopify-migration@hdc-digital
/plugin install shopify-design@hdc-digital
/plugin install shopify-abnahme@hdc-digital
```

Danach Claude Code neu starten oder `/reload-plugins` ausführen. Die Plugins stehen ab dann in **jedem** Projektordner zur Verfügung. Updates kommen automatisch beim nächsten Marketplace-Abgleich.

Voraussetzung: Zugriff auf das GitHub-Repo, einmalig `gh auth login`.

Custom App — einmal je Kunde

Der Token wird **nicht** von Claude erzeugt. Das macht ein Mensch im Shop-Admin. Es gibt zwei Wege, und die Verwechslung kostet regelmäßig eine halbe Stunde:

Weg	Erkennungsmerkmal	Ergebnis
Shop-Admin Einstellungen → Apps → Apps entwickeln	Kein Feld „Zulässige Weiterleitungs-URL(s)“	Der Token steht direkt da. Das ist der Normalfall.
Partner Dashboard	Es gibt ein Feld „Zulässige Weiterleitungs-URL(s)“	Kein direkter Token — es braucht einen OAuth-Durchlauf. Nur nehmen, wenn der erste Weg nicht möglich ist.

Die vollständige Anleitung mit allen Klicks steht in `shopify-migration/references/custom-app.md`, die Liste der nötigen Berechtigungen in `shopify-settings/references/berechtigungen.md`. Sie deckt alle Skills ab.

Der Token gehört in eine Datei

```
~/config/shopify-<kunde>.env

SHOP=kunde-xxxx.myshopify.com
TOKEN=shpat_...
```

Zugangsdaten nie in den Chat

Token und Client Secret gehören nicht in den Chat, nicht in E-Mails, nicht in Tickets und nicht auf Screenshots. Claude liest die Datei selbst. Ist ein Secret trotzdem einmal sichtbar geworden, muss es im Dev Dashboard rotiert werden — und anschließend der Token neu geholt werden, weil die Rotation ihn ungültig macht.

Fehlende Berechtigungen sind kein Blocker

Alle Prüfungen laufen auch mit unvollständigen Rechten. Sie melden dann konkret, was fehlt, und der Punkt gilt als *ungeprüft*. Häufig fehlen `read_legal_policies`, `read_shipping`, `read_locations` und `read_apps`.

Grundeinrichtung

shopify-settings — richtet den Rahmen ein: Stammdaten, Richtlinien, Versand, Sprachen, Märkte, Standorte, Steuern, Metafeld-Definitionen. Noch keine Produkte.



Zugang

Token einlesen, Berechtigungen prüfen. Fehlen welche, laufen die Prüfungen trotzdem und melden, was nicht sichtbar ist.



Bestandsaufnahme

Prüft Stammdaten, Sichtbarkeit, Richtlinien, Versand, Sprachen, Märkte, Standorte, Steuern und Metafelder. Der Befund wird im Chat zusammengefasst und trennt dabei drei Dinge: was blockiert, was geprüft gehört, und was mangels Berechtigung unklar blieb.

Erfahrungsgemäß lohnt der Blick auf: Zeitzone (Dev-Shops stehen oft auf US-Zeit), Shop-Adresse (steht oft noch auf der Agentur), Versandtarife (ohne Tarif kann niemand bestellen), fehlende Richtlinien.

```
1_bestandsaufnahme.py → Einrichtungsstand.md
```



Freigabe 1 — Befund besprochen, Reihenfolge festgelegt



Rechtliches

Fehlende Richtlinien werden *angefordert*, nicht geschrieben. Liegen Texte vor, werden sie eingespielt und anschließend in der Storefront geprüft: erreichbar und im Footer verlinkt?

```
2_richtlinien.py --datei agb.html --typ TERMS_OF_SERVICE
```



Versand

Versandzonen und Tarife brauchen konkrete Werte vom Kunden: welche Länder, welcher Preis, ab welchem Warenwert versandkostenfrei. Ohne diese Angaben wird nichts angelegt — ein geschätzter Versandpreis ist eine Zusage an Käufer.



Sprachen, Märkte, Metafelder

Metafeld-Definitionen entstehen **vor** der Produktmigration — sonst fehlen sie später bei allen Produkten. Wichtig dabei: `access.storefront = PUBLIC_READ`, sonst sind sie im Theme unsichtbar, obwohl sie im Admin gefüllt aussehen.



Was nur von Hand geht

Zahlungen, Domain, Checkout-Branding, Kundenkonten, E-Mail-Vorlagen, Plan. Claude führt durch den Shop-Admin. Steht der Browser zur Verfügung, kann es Formulare selbst ausfüllen — der Shopify-Admin steckt allerdings in Shadow DOM, Werte müssen getippt und nicht zugewiesen werden.

Zahlungsanbieter nie selbst einrichten. Dort werden Bankdaten und Ausweise verlangt.



Freigabe 2 — Checkliste abgearbeitet



Abschluss

Bestandsaufnahme erneut laufen lassen und mit dem ersten Lauf vergleichen. Was ist erledigt, was bleibt offen, was liegt beim Kunden.

Aufbau im Theme

shopify-design — elf Phasen, drei Freigaben. Zwei Dokumente werden abgenommen, bevor das Theme überhaupt angefasst wird: erst die Copy, dann der Startseiten-Entwurf.

Warum zwei Abnahmen vor dem Theme

Die Copy beantwortet, *was* da steht. Der Entwurf beantwortet, *wie es wirkt*. Beides gleichzeitig zu klären funktioniert nicht — über Farben zu diskutieren, während der Text noch wackelt, kostet zwei Runden statt einer. Steht beides, ist der Theme-Aufbau Ausführung statt Entwurf.

0

Schlüssel

Einmal je Rechner. Claude *zeigt* die Ablage, nimmt den Schlüssel nicht entgegen — er gehört in `~/ .config/openai.env`, nicht in den Chat. `--schluessel-pruefen` kostet nichts und unterscheidet einen falsch kopierten Schlüssel von einer nicht verifizierten Organisation.

```
8b_bild_erzeugen.py --schluessel-pruefen
```

1

Unterlagen sichten

Scope-Dokument und Konzeptpräsentation. Fehlt eines von beidem, wird das gesagt statt geraten.

2

Konzept auswerten

Liest Struktur, Abschnitte und Farbwerte aus — inklusive der Füllfarben von Vektorflächen in der Präsentation.

```
1_konzept_lesen.py → konzept.json
```

3

Shop-Copy schreiben

Als Senior Copywriter, nicht als Platzhalterfüller: Zielgruppe aus dem Konzept ableiten, konversionsstark und zielgruppengerecht schreiben, das Dokument vollständig füllen. Offene Stellen als [RÜCKFRAGE ...], nicht plausibel erfunden.

```
2_copy_gueruest.py · 3_copy_ansicht.py → Shop-Copy.html
```

✓

Freigabe 1 — Shop-Copy vom Kunden abgenommen

4

Startseiten-Entwurf

Eine gestaltete HTML-Seite, die zeigt, wie die Startseite aussehen wird. Kein Theme, kein Klickdummy, genau eine Seite — sie trägt die Gestaltungsentscheidung, alles Weitere folgt ihr. Aus ihrer Abschnittsfolge entstehen später die Sections.

Das Gerüst zieht die Abschnitte aus der freigegebenen Copy und legt Palette und Schriften an. Danach wird gestaltet — eigene Typografie, echte Bilder, Abschnittsrhythmus. **Offene Punkte als Fußnote am Abschnitt**, nicht am Ende: Beim Gestalten fällt fast immer etwas auf, das vorher niemand gesehen hat, etwa zwei widersprüchliche Versandregeln.

Die Prüfung endet mit Fehlercode bei Blindtext, Platzhaltern oder fehlender Entwurfs-Kennzeichnung. Bildplatzhalter für noch fehlende Kundenfotos sind dagegen in Ordnung — eine Lieferung, kein Mangel.

```
3b_entwurf.py --geruest · --pruefen → Entwurf-Startseite.html
```

✓

Freigabe 2 — Entwurf abgenommen. Erst danach wird das Theme angefasst.

5

Theme inventarisieren

Liest aus, welche Sections und Blöcke *dieses* Theme wirklich hat. Ein Feldname aus einem anderen Projekt wird stillschweigend ignoriert.

```
4_theme_inventar.py
```

6

Aufbau

Reihenfolge: erst Schriften und Farbpalette, dann die Bilder, dann die Sections aus dem Entwurf, dann Header, Footer und die Templates. Gearbeitet wird immer auf einem Theme-**Duplikat**; die Skripte verweigern das aktive Theme.

Der Aufbau zerfällt in drei sichtbare Schritte: **Gestaltung** (Schriften und Farbpalette), **Bilder** (Hero, Kategoriebanner, Motive — siehe nächstes Kapitel) und **Sections**, die aus dem freigegebenen Entwurf entstehen, in derselben Reihenfolge.

```
7_gestaltung.py · 8_ 8b_ 8c_ 8d_ · 5_aufbau.py
```

7

Demo-Produkt

Vor der Migration, nicht danach. An einem einzigen Produkt wird die ganze Kette geprüft: Definition da → Wert gesetzt → Storefront-Zugriff offen → Theme gibt es aus.

Der Fehler, den dieser Schritt abfängt, ist immer derselbe: Ein Metafeld ist im Admin gefüllt und im Theme unsichtbar, weil `access.storefront` nicht auf `PUBLIC_READ` steht. Bei einem Produkt kostet das zehn Minuten. Bei dreihundert kostet es einen Tag.

Das Produkt liegt als Entwurf mit Tag `hdc-demo` im Shop und wird vor dem Livegang gelöscht — der Punkt steht auch in der Übergabe.

```
13_demoprodukt.py --anlegen · --entfernen
```



Theme-Einstellungen

Schritt 2 der HDC-Checkliste: Logo, Favicon, Quick-View aus, zweites Bild bei Mouseover, Warenkorb als Einschub, Bestellhinweis an, Express-Checkout aus, „Powered by Shopify“ raus. Das Skript liest erst das settings_schema und arbeitet nur mit Schaltern, die es dort gibt.

```
9_checkliste.py --logo logo.png --favicon favicon.png --setzen
```



Kategorie-, Produkt- und Serviceseiten

Schritt 6 bis 8 der Checkliste. Was eine Zusage an Käufer enthielte – Lieferzeit, Versandkosten, Rückgabe – kommt als [RÜCKFRAGE ...] hinein. Serviceseiten bleiben **unveröffentlicht**, bis die Rückfragen beantwortet sind.

```
10_kategorieseite.py · 11_produktsseite.py · 12_serviceseiten.py
```



Prüfen

Offene Rückfrage-Marker, Platzhaltertexte, Sections ohne Kollektion, nicht gesetzte Bilder, unangepasste Standardlinks. Danach der Browserdurchgang.

```
6_pruefung.py
```



Freigabe 3 — Aufbau abgenommen

Bilder

Vier Werkzeuge, fünf Bildsorten, zwei harte Grenzen. Ein Shop ohne Bilder ist eine Präsentationsfolie — und zwei Bildsorten dürfen trotzdem nie erzeugt werden.

Welches Werkzeug wofür

Skript	Bildsorte	Herkunft
<code>8_bilder.py</code>	Fläche und Form	HTML-Vorlage, hier gerendert. Verläufe, Bühnen, geometrische Motive — und Kompositionen aus Kundenfoto plus Fläche, mit Freisteller über die macOS-Vision-Bibliothek
<code>8b_bild_erzeugen.py</code>	Einzelmotiv	Erzeugt Texturen, Stimmungen, freigestellte Elemente. <code>-transparent</code> liefert einen echten Alphakanal und geht ohne Umweg als <code>--foto</code> in die Vorlagen
<code>8c_hero.py</code>	Der Hero	Erzeugt nach den HDC-Hero-Regeln, aus einem JSON-Prompt
<code>8d_kategoriebanner.py</code>	Bannersatz	Erzeugt als Satz — ein Stil für alle Kollektionen, nur das Motiv unterscheidet sich

Der Hero

Das einzige Bild, das jeder Besucher sieht. Er entsteht aus einem Interview, nicht aus einem Halbsatz.

- Was wird verkauft?
- Wer nutzt es, in welcher Situation, an welchem Ort?
- Läuft eine Aktion — Rabatt, Saison, Neueinführung?
- Was genau soll beworben werden: das Sortiment, ein Produkt, ein Versprechen?
- Gibt es Beispielbilder oder Referenzshops?
- Welcher Stil — warm und wohnlich, technisch und klar, laut und werblich?

Erst danach drei Szenen auf Deutsch, die sich wirklich unterscheiden — in Ort, Tageszeit oder Blickwinkel, nicht drei Varianten derselben Idee. Nach der Auswahl der JSON-Prompt auf Englisch.

Feste Regel	Warum
3200×900	Vollflächiger Streifen über die ganze Breite
Linke Hälfte frei	Dort steht die Headline. Dezentos Overlay, hell oder dunkel je nach Textfarbe
Alles Visuelle rechtsbündig	Sonst kollidiert das Motiv mit dem Text
Realistische Nutzungsszene	Kein Studiofreisteller — der Hero zeigt eine Situation, kein Produktfoto

Das Skript hängt diese Regeln bei jedem Lauf selbst an den Prompt — sie lassen sich nicht versehentlich weglassen.

Kategoriebanner

Die sieht niemand einzeln. Wer sich durch den Shop klickt, sieht vier hintereinander. Sie müssen als **Satz** wirken: gleiches Licht, gleiche Kamerahöhe, gleiche Farbwelt — und nur das Motiv unterscheidet sich. Deshalb eine Stil-Datei für alle.

- -pruefen sagt, welche Kollektionen kein Banner haben und für welche noch das Motiv fehlt. **Ein Motiv erfindet das Skript nicht** — was in einer Kategorie zu sehen ist, weiß der Kunde. Vor dem Setzen alle nebeneinander ansehen: Ein Satz, bei dem eines aus der Reihe fällt, wirkt schlechter als gar keiner.

Zwei Bildsorten werden nie erzeugt

Das Produkt des Kunden.

Ein erzeugtes Produktbild zeigt Ware, die es so nicht gibt — irreführende Werbung, und zwar die Art, die auffällt, wenn das Paket ankommt.

Menschen, die es wirklich gibt.

Ein erzeugtes Gesicht auf einer „Über uns“-Seite behauptet einen Menschen, den es nicht gibt.

Soll das Produkt im Bild sein, kommt es als echtes Foto über `referenced_image_ids` hinein oder wird nachträglich einkomponiert.

Grenzen des Modells

Empirisch ermittelt, nicht aus der Dokumentation übernommen.

Grenze	Folge
Beide Seiten durch 16 teilbar	900 geht nicht, 896 schon
Längste Kante höchstens 3840	Genug für jeden Hero
Seitenverhältnis höchstens 3:1	3200×900 ist 3,56:1 und damit zu breit. Das Skript erzeugt 3200×1072 und schneidet auf das Band herunter — reiner Zuschchnitt, kein Hochskalieren

Modelle auf dem Schlüssel: `gpt-image-2` als Standard und das einzige mit freien Größen, daneben `gpt-image-2-2026-04-21`, `gpt-image-1.5`, `gpt-image-1`, `gpt-image-1-mini` und `chatgpt-image-latest`.

Ein Hero in 3200×896 kostete im Test 4656 Token bei Qualität high, ein Satz aus drei Kategoriebannern 11 188 Token. Das Skript gibt den Verbrauch nach jedem Lauf aus — den Eurobetrag hat nur das Dashboard.

Produktmigration

shopify-migration — sieben Phasen, fünf Freigaben. Die dichteste Freigabe-Folge im ganzen Prozess, weil ein fehlerhafter Import hunderte Datensätze betrifft.



Zugang

Custom App und Token wie in Kapitel 3. Claude erzeugt sie nicht.



Freigabe 1 — Zugang steht



Browser

Nicht optional. Claude in Chrome läuft bei jeder Migration mit. Was die API meldet und was im Shop steht, ist nicht dasselbe: Eine Kollektion kann laut API gefüllt sein und in der Storefront leer aussehen, ein Metafeld gesetzt und unsichtbar, ein Bild hochgeladen und dem falschen Produkt zugeordnet.

Genutzt in drei Momenten: vor dem Import den Zielshop ansehen, nach dem Probelauf die ersten Produkte in der Storefront prüfen, bei der Abnahme Stichproben über Kategorie, Produktseite und Warenkorb.



Shop verstehen

Der Steckbrief liest den Zielshop aus: welche Metafelder gibt es, welche Produktoptionen, welche Tag-Struktur, welche Kollektionen und nach welchen Regeln füllen sie sich. Ohne diesen Schritt landen Produkte in leeren Kategorien, weil ein Tag `beutel` und `rollen` heißt und die Kollektion `beutel & rollen` erwartet.

```
1_steckbrief.py
```



Freigabe 2 — Tag-Vokabular bestätigt



Vorlage füllen

Die Importvorlage wird passend zu *diesem* Shop erzeugt — mit genau den Spalten, die er kennt. Beim Füllen gilt: Fehlende Maße, Gewichte, Preise und Herstelleranschriften werden nicht ergänzt, sondern zur Rückfrage.

```
2_vorlage.py
```



Prüfen

Vor dem Import: Pflichtfelder, doppelte SKUs, Handle-Kollisionen, Varianten, die sich gegenseitig ausschließen, Bildzuordnungen. Der Lauf endet mit Fehlercode, solange Blocker offen sind.

`3_pruefen.py`



Freigabe 3 — Prüfung fehlerfrei



Import

Zuerst ein Probelauf mit wenigen Produkten, dann der Rest. Der Import ist nach einem Abbruch wieder aufnehmbar — bereits angelegte Produkte werden übersprungen statt doppelt erzeugt.

`4_import.py`



Freigabe 4 — Probelauf in Ordnung



Abnahme

Soll-Ist-Vergleich: Anzahl Produkte, Varianten, Bilder, Kollektionszuordnung. Smart Collections füllen sich bei Shopify verzögert — direkt nach dem Import wirken sie leer, obwohl alles stimmt.

`5_abnahme.py`



Freigabe 5 — Abnahme besprochen

Abnahme

shopify-abnahme führt drei Fachprüfungen nacheinander aus, setzt um was maschinell geht und schreibt die Übergabe. Alle Befunde laufen in denselben Dateien zusammen.



Alle drei Prüfungen

Startet die Skripte aus `shopify-ux`, `shopify-cro` und `shopify-recht` und schreibt `befunde/ux.json`, `befunde/cro.json`, `befunde/recht.json`. Am Ende steht der Stand: wie viele Blocker, wie viele ungeprüft.

```
abnahme.py --theme <duplikat-id>
```



Der Durchgang

Die Skripte finden die *messbare* Hälfte. Die andere Hälfte findet nur, wer den Shop benutzt. Drei Fachbrillen nacheinander, Beobachtungen kommen in `befunde/durchgang.json` — gleiche Form, gleiches Gewicht.



Freigabe 1 — Befund vollständig, bevor irgendetwas geändert wird



Umsetzen

Setzt um, wofür es einen hinterlegten Handgriff gibt. Alles andere bleibt liegen — auch wenn ein Befund sich selbst als automatisch umsetzbar markiert; das meldet das Skript ausdrücklich. Lieber ein offener Punkt zu viel als ein falsch abgehakter.

```
umsetzen.py --setzen
```



Übergabe

Schreibt `uebergabe.md`, sortiert nach Zuständigkeit. Wird als Artifact veröffentlicht und dem Kunden gegeben.

```
uebergabe.py
```



Freigabe 2 — Übergabe abgenommen

Die drei Fachbrillen

shopify-ux · Benutzerführung

Drei Kaufwege, immer dieselben.

„Ich weiß was ich will“ · „Ich weiß es noch nicht“ · „Ich habe eine Frage“. Klicks zählen, notieren wo man stockt. Genau da liegt der Befund.

Gemessen: WCAG-Kontraste, Startseitenlänge, Navigationsbreite, tote Menüpunkte, Alt-Texte, dünne Beschreibungen.

shopify-cro · Verkaufspsychologie

Fünf Einwände in fester Reihenfolge.

Bin ich hier richtig → Ist das das Richtige → Kann ich denen glauben → Was kostet es wirklich → Was, wenn es nicht passt. Eine Frage, die erst im Checkout beantwortet wird, ist zu spät beantwortet.

Gezählt: Bilder je Produkt, leere Kategorien, Bewertungen, Empfehlungen, Zahlungssicons, Newsletter, Warenkorb-Typ.

shopify-recht · Pflichtangaben

Stellt fest, *ob* etwas da ist — nicht, ob der Text trägt.

Richtlinien, MwSt- und Versandkostenhinweis, Grundpreis nach PAngV, GPSR-Metafelder, Cookie-Einwilligung, Versandtarife, Rechtstexte in allen veröffentlichten Sprachen.

Schreibt keine Rechtstexte und gibt keine Rechtsauskunft.

Das Befund-Format

Ein Format für Skript und Mensch.

Jeder Punkt trägt Schwere (blocker, wichtig, kosmetik), Ort, Befund, Empfehlung und wer es umsetzen kann. Claude trägt dort auch ein, was es im Durchgang selbst gefunden hat — dieselbe Datei, dasselbe Gewicht.

Mobil wird nur behauptet, wenn es geprüft wurde

Das Browserfenster zu verkleinern reicht nicht — der Rendering-Viewport bleibt breit, und man prüft gegen die Desktop-Ansicht, ohne es zu merken. Ohne echte Geräteemulation wird das offen gesagt und die Prüfung an einen Menschen mit Handy abgegeben. Eine behauptete Mobilprüfung ist schlimmer als keine — sie schließt den Punkt, ohne ihn zu prüfen.

Die Übergabe

Ein Dokument, sortiert nach Zuständigkeit statt nach Thema. Der Wert liegt darin, dass man ihm glauben kann.

Aufbau von Uebergabe.md

Nr.	Abschnitt	Was drinsteht
1	Blocker vor dem Livegang	Ohne das geht der Shop nicht live. Die rechtlichen Punkte stehen hier, nicht unter „wäre schön“.
2	Liegt beim Kunden	Angaben, Texte, Fotos, Entscheidungen. Geschätzte Werte wären hier eine Zusage an Käufer.
3	Von Hand im Shop-Admin	Technisch machbar, aber nicht über die Schnittstelle erreichbar.
4	Immer vor dem Livegang	Die Punkte, die jeden Shop betreffen — siehe unten.
5	Erledigt	Mit dem, was tatsächlich gemacht wurde. Nicht „umgesetzt“, sondern was.
6	Nicht prüfbar	Fehlende Berechtigungen. Diese Punkte sind <i>nicht</i> in Ordnung, sondern unbekannt.

Immer vor dem Livegang

Unabhängig vom Befund. Diese Liste steht in jeder Übergabe.

Punkt	Zuständig	Warum
Zahlungsanbieter aktiv und getestet	Kunde	Bankdaten und Ausweis gehen nur über den Shop-Inhaber. Danach eine echte Testbestellung.
Domain verbunden und primär	Kunde	Solange die myshopify-Adresse primär ist, laufen Links und Tracking darauf.
Bezahlter Plan gebucht	Kunde	Ein Entwicklungsshop kann nicht verkaufen.
Passwortschutz aufheben	Agentur	Zum Schluss. Vorher ist der Shop für Suchmaschinen und Neugierige zu.
Testbestellung durchgeführt	Agentur	Bestellung, Bestätigungsmail, Rechnung, Storno. Einmal komplett.
E-Mail-Vorlagen auf das Branding angepasst	Agentur	Bestellbestätigung und Versandbenachrichtigung sind die meistgelesenen Seiten des Shops.
Weiterleitungen der alten URLs	Agentur	Nur bei einem Relaunch. Ohne sie fällt die alte Sichtbarkeit weg.

Wie das Ergebnis vorgelegt wird

Als Artifact veröffentlichen, Link weitergeben. Im Chat drei Sätze: wie viele Blocker, was der Kunde liefern muss, was als Nächstes ansteht.

Die schlechte Nachricht zuerst.

Wenn der Shop in zwei Wochen live soll und noch sechs Blocker offen sind, ist das der erste Satz — nicht der letzte.

Anhang

Die alte Checkliste, übersetzt

Wo die bisherigen Schritte der HDC-Checkliste jetzt liegen.

Schritt der Checkliste	Skill	Grad der Automatisierung
0 · Theme-Auswahl, Zielgruppe, Konkurrenz	shopify-design Ph. 1–2	Auswertung automatisch, Theme-Entscheidung beim Menschen
1 · Shop-Details, Währung, Versand, Steuern, Standorte, Märkte, Sprachen	shopify-settings Ph. 2–5	weitgehend automatisch
1 · Checkout-Branding, Kundenkonten, Benachrichtigungen	shopify-settings Ph. 6	geführt, von Hand
2 · Theme-Einstellungen	shopify-design Ph. 8	automatisch, soweit das Theme die Schalter hat
3 · Apps	—	installiert immer ein Mensch
4 · Produkte und Medien	shopify-migration	automatisch, Bildqualität wird geprüft und gemeldet. Läuft nach dem Aufbau — die Metafelder sind am Demo-Produkt geprüft
<i>neu</i> · Website-Konzept als HTML	shopify-design Ph. 4	Gerüst automatisch, Gestaltung durch Claude, Abnahme durch den Kunden
<i>neu</i> · Hero und Kategoriebanner	shopify-design Ph. 6	erzeugt; Motive und Produktfotos kommen vom Kunden
<i>neu</i> · Demo-Produkt für die Metafelder	shopify-design Ph. 7	automatisch, Sichtprüfung im Theme durch einen Menschen
5 · Startseite	shopify-design Ph. 4 + 8	Website-Konzept, daraus die Sections
6 · Kategorieseite	shopify-design Ph. 9	automatisch; Sortierung nach Bestand nur bei manuellen Kollektionen
7 · Produktseite	shopify-design Ph. 9	Bausteine automatisch; Bewertungen, Größentabellen, Bundles brauchen Apps
8 · Service-Seiten	shopify-design Ph. 9	Gerüst automatisch, Inhalte vom Kunden

<i>neu</i> · UX-, CRO- und Rechtsprüfung	shopify-abnahme	messbarer Teil automatisch, Durchgang durch Menschen
<i>neu</i> · Übergabe-Checkliste	shopify-abnahme Ph. 4	vollständig automatisch aus den Befunden

Fallen, die regelmäßig Zeit kosten

- **Metafeld unsichtbar im Theme.** Die Definition braucht `access.storefront = PUBLIC_READ`. Ohne das ist das Feld im Admin gefüllt und in der Storefront nicht vorhanden. Genau dafür gibt es das Demo-Produkt — der Fehler kostet dort zehn Minuten statt einen Tag.
- **Kollektion bleibt leer.** Tag-Schreibweise vergleichen — ein und statt & reicht. Und: Smart Collections berechnet Shopify verzögert, direkt nach dem Import wirken sie leer.
- **Schalter ohne Wirkung.** `settings_data.json` nimmt jeden Schlüssel an. Steht er nicht im `settings_schema` des Themes, wird er beim Rendern ignoriert — kein Fehler, keine Meldung.
- **null heißt nicht „aus“.** Es heißt „nie angefasst“, dann gilt der Standard aus dem Schema. Der kann `true` sein.
- **Direkt nach dem Schreiben gelesen.** Die Asset-API liefert kurz nach einem Schreibvorgang noch die alte Fassung. Ein paar Sekunden warten.
- **Menüpunkt vom Typ Produkt.** Bindet an die Produkt-ID, nicht an den Handle. Ein umbenannter Handle repariert den Link nicht.
- **Bilder sind nicht pro Sprache übersetzbar.** Nur der Alt-Text ist es. Sprachabhängige Bilder gehen ausschließlich über einen Filter im Theme.
- **Skripte nicht auffindbar.** Bei lokaler Installation liegt in `~/ .claude/skills/` ein Symlink. `find` folgt dem im Standard nicht — es braucht `find -L`.

Wo was liegt

```

hdc-shopify-migration/
├─ .claude-plugin/marketplace.json      Marketplace-Definition
├─ werkzeuge/lokal-installieren.sh      Skills per Symlink nach ~/ .claude/skills/
├─ plugins/<plugin>/
│   ├─ .claude-plugin/plugin.json       Plugin-Manifest
│   └─ skills/<skill>/
│       ├─ SKILL.md                     führt das Gespräch
│       ├─ references/                  Claude liest, gibt im Chat wieder
│       ├─ vorlagen/                   HTML-Vorlagen für Bildmotive
│       └─ scripts/                     nummeriert in Ablaufreihenfolge

```

`scripts/shopify.py` liegt bewusst in jedem Skill als Kopie — Skills sollen einzeln lauffähig sein. Änderungen daran gehören in alle Kopien.

Warum curl statt Python-Bordmitteln

Die Python-Standardbibliothek scheitert auf einigen Rechnern an der Zertifikatsprüfung. Alle Shopify-Aufrufe laufen deshalb über `curl` als Unterprozess. Das ist Absicht, kein Provisorium.

